Studio Nx

JS 129 — nextTrackNumber()

2026-10-14

JS 129 — `nextTrackNumber()` (새 트랙 번호)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 열다섯 번째 함수 (FR-207 트랙 추가)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	트랙 삭제 뒤 번호를 다시 매기는 `renumberMap()` 함수를 이어서 진행합니다

1. 하는 일

트랙을 새로 추가할 때 **붙일 번호**를 돌려줍니다. 번호 = 지금 트랙 번호 중 **가장 큰 값 + 1**입니다.

> 새 번호 = (지금까지 가장 큰 번호) + 1, 트랙이 없으면 1

지금 번호	새 번호	이유
`[]`	1	트랙 없음
`[1, 2, 3]`	4	가장 큰 값 3 + 1
`[1, 3]`	4	2가 비어도 가장 큰 값 기준
`[3, 1, 2]`	4	순서와 무관
`[100, 3]`	101	가장 큰 값 100 + 1
`[1, null, 2]`	3	그룹 트랙(번호 없음)은 건너뜀

```
1. Kick   2. Bass   3. Lead   →  [+ Track]  4. ...
```

2. 코드

```
export function nextTrackNumber(numbers) {
  if (!Array.isArray(numbers)) throw new RangeError('numbers 는 배열이어야 합니다');
  let max = 0;
  for (const n of numbers) {
    if (typeof n === 'number' && isFinite(n) && n > max) max = n;
  }
  return max + 1;
}
```

최신 문법으로 쓰면 (공부용)

```
export const nextTrackNumber = (numbers) => Math.max(0, ...numbers.filter(Number.isFinite)) + 1;
```

최신 문법	실제 코드	뜻
`Math.max(0, ...arr)` 스프레드	`for` 반복	같은 결과입니다. 목록이 아주 길 때 스프레드가 인자 한계에 걸릴 수 있어 반복으로 씁니다
`Number.isFinite`	`isFinite`	둘 다 되지만 여기선 `typeof`까지 함께 봅니다

3. 왜 이렇게

선택	이유
가장 큰 값 + 1	프로토타입 `createTrack()`이 쓰던 규칙 그대로입니다. 중간 번호가 비어도(삭제 전 상태) 겹치지 않게 붙습니다
번호 없는 줄은 셈하지 않음	그룹 트랙처럼 번호가 없는 줄이 끼어도 번호가 밀리지 않습니다
`for` 반복	`Math.max(0, …numbers)`보다 안전하고, 숫자가 아닌 값도 자연스럽게 걸러집니다
배열이 아니면 오류	트랙 목록을 잘못 넘기면 조용히 1을 돌려줘 번호가 겹칩니다. 바로 알립니다

4. 시험 (Node에서 직접 실행, 12개 통과)

확인	결과
없음 → 1 · [1,2,3] → 4 · [1,3] → 4 · [7] → 8	✓
중복([1,2,2]→3) · 순서 뒤섞임([3,1,2]→4)	✓
그룹 줄 건너뜀([1,null,2]→3) · 0은 안 셈([0]→1)	✓
문자열 무시(["5",1]→2) · 큰 번호([100,3]→101)	✓
배열 아님·없음 → 오류	✓